

Lab 1.2: Send and Receive Messages

Course 1: LoRa & LoRaWAN Fundamentals

USP Training Series

1 Lab Overview

Lab Objectives

By the end of this lab, you will be able to:

- Send unconfirmed uplink messages
- Format and encode payloads
- Receive and decode downlink messages
- Understand FPort usage
- Monitor message delivery in network server

Prerequisites

Required: Completion of Lab 1.1 (Basic LoRaWAN Join)

Hardware: Xiao nRF54L15 + LR1120 board

2 Lab Duration

Estimated Time: 45 minutes

3 Background

3.1 LoRaWAN Ports

FPort (Frame Port) identifies the application endpoint:

- Port 0: Reserved for MAC commands
- Ports 1-223: Application data
- Port 224-255: Reserved for future use

3.2 Payload Encoding

Best practices for payload encoding:

- Keep payloads small (< 51 bytes for SF12, < 222 bytes for SF7)
- Use binary encoding instead of ASCII
- Consider byte order (big-endian vs little-endian)
- Use The Things Network payload formatters for decoding

4 Lab Instructions

4.1 Step 1: Implement Uplink Function

Create a function to send sensor data as unconfirmed uplink.

4.2 Step 2: Encode Sensor Data

Pack temperature, humidity, and battery voltage into compact payload.

4.3 Step 3: Send Periodic Uplinks

Use Zephyr timers to send data every 60 seconds.

4.4 Step 4: Receive Downlinks

Implement callback to handle received downlink messages.

4.5 Step 5: Test Communication

Build, flash, and verify bidirectional communication.

5 Solution

Solution

5.1 Complete Implementation

```
1 #include <zephyr/kernel.h>
2 #include <zephyr/logging/log.h>
3 #include <zephyr/lorawan/lorawan.h>
4 #include <zephyr/random/random.h>
5
6 LOG_MODULE_REGISTER(lab_1_2, LOG_LEVEL_INF);
7
8 /* Simulated sensor readings */
9 static int16_t get_temperature(void)
10 {
11     /* Simulate temperature: 20.5 C = 205 (x10) */
12     return 205 + (sys_rand32_get() % 100) - 50;
13 }
14
15 static uint8_t get_humidity(void)
16 {
17     /* Simulate humidity: 60% */
18     return 60 + (sys_rand32_get() % 40);
19 }
20
21 static uint16_t get_battery_voltage(void)
22 {
23     /* Simulate battery: 3.6V = 3600mV */
24     return 3600 + (sys_rand32_get() % 200);
25 }
26
27 /* Encode sensor data into compact payload */
28 static int encode_sensor_data(uint8_t *payload)
```

```

29 {
30     int16_t temp = get_temperature();
31     uint8_t hum = get_humidity();
32     uint16_t vbat = get_battery_voltage();
33
34     /* Payload format (6 bytes):
35      * [0-1]: Temperature (int16, x10, big-endian)
36      * [2]: Humidity (uint8, %)
37      * [3-4]: Battery voltage (uint16, mV, big-endian)
38      * [5]: Sequence counter
39      */
40     static uint8_t seq = 0;
41
42     payload[0] = (temp >> 8) & 0xFF;
43     payload[1] = temp & 0xFF;
44     payload[2] = hum;
45     payload[3] = (vbat >> 8) & 0xFF;
46     payload[4] = vbat & 0xFF;
47     payload[5] = seq++;
48
49     LOG_INF("Encoded: Temp=%.1f C , Hum=%d%%, Vbat=%dmV",
50             temp / 10.0, hum, vbat);
51
52     return 6;
53 }
54
55 /* Send uplink message */
56 static void send_uplink(void)
57 {
58     uint8_t payload[6];
59     int len;
60     int ret;
61
62     len = encode_sensor_data(payload);
63
64     ret = lorawan_send(2, payload, len,
65                       LORAWAN_MSG_UNCONFIRMED);
66     if (ret < 0) {
67         LOG_ERR("Send failed: %d", ret);
68     } else {
69         LOG_INF("Uplink sent (%d bytes)", len);
70         LOG_HEXDUMP_INF(payload, len, "Payload");
71     }
72 }
73
74 /* Downlink callback */
75 static void dl_callback(uint8_t port, bool data_pending,
76                        int16_t rssi, int8_t snr,
77                        uint8_t len, const uint8_t *data)
78 {
79     LOG_INF("=====");
80     LOG_INF("DOWNLINK RECEIVED");
81     LOG_INF("=====");
82     LOG_INF("Port: %d", port);
83     LOG_INF("RSSI: %d dBm", rssi);
84     LOG_INF("SNR: %d dB", snr);

```

```

85     LOG_INF("Pending: %s", data_pending ? "Yes" : "No");
86     LOG_INF("Length: %d bytes", len);
87
88     if (len > 0) {
89         LOG_HEXDUMP_INF(data, len, "Data");
90
91         /* Example: Parse simple command */
92         if (port == 2 && len >= 1) {
93             switch (data[0]) {
94                 case 0x01:
95                     LOG_INF("Command: LED ON");
96                     break;
97                 case 0x00:
98                     LOG_INF("Command: LED OFF");
99                     break;
100                case 0xFF:
101                    LOG_INF("Command: Reset");
102                    break;
103                default:
104                    LOG_INF("Command: Unknown (0x%02X)", data[0]);
105                    break;
106            }
107        }
108    }
109 }
110
111 /* Uplink timer */
112 static void uplink_timer_handler(struct k_timer *timer)
113 {
114     send_uplink();
115 }
116
117 K_TIMER_DEFINE(uplink_timer, uplink_timer_handler, NULL);
118
119 int main(void)
120 {
121     int ret;
122     const struct device *lora_dev;
123
124     LOG_INF("=====");
125     LOG_INF("Lab 1.2: Send and Receive Messages");
126     LOG_INF("=====");
127
128     lora_dev = DEVICE_DT_GET(DT_ALIAS(lora0));
129     if (!device_is_ready(lora_dev)) {
130         LOG_ERR("LoRa device not ready");
131         return -1;
132     }
133
134     ret = lorawan_start();
135     if (ret < 0) {
136         LOG_ERR("Failed to start LoRaWAN: %d", ret);
137         return ret;
138     }
139
140     /* Register downlink callback */

```

```

141     struct lorawan_downlink_cb downlink_cb = {
142         .port = LW_RECV_PORT_ANY,
143         .cb = dl_callback
144     };
145     lorawan_register_downlink_callback(&downlink_cb);
146
147     /* Join network */
148     struct lorawan_join_config join_cfg = {
149         .mode = LORAWAN_ACT_OTAA,
150         .dev_eui = (uint8_t[]){0x00,0x00,0x00,0x00,
151                                0x00,0x00,0x00,0x01},
152         .otaa = {
153             .join_eui = (uint8_t[]){0x00,0x00,0x00,0x00,
154                                     0x00,0x00,0x00,0x00},
155             .app_key = (uint8_t[]){0x00,0x00,0x00,0x00,
156                                    0x00,0x00,0x00,0x00,
157                                    0x00,0x00,0x00,0x00,
158                                    0x00,0x00,0x00,0x00},
159             .nwkey = (uint8_t[]){0x00,0x00,0x00,0x00,
160                                  0x00,0x00,0x00,0x00,
161                                  0x00,0x00,0x00,0x00,
162                                  0x00,0x00,0x00,0x00}
163         }
164     };
165
166     LOG_INF("Starting join procedure...");
167     ret = lorawan_join(&join_cfg);
168     if (ret < 0) {
169         LOG_ERR("Join failed: %d", ret);
170         return ret;
171     }
172
173     /* Wait for join */
174     while (!lorawan_is_joined()) {
175         k_sleep(K_SECONDS(1));
176     }
177
178     LOG_INF("Network joined successfully!");
179
180     /* Send first uplink immediately */
181     send_uplink();
182
183     /* Start periodic uplinks (60 seconds) */
184     k_timer_start(&uplink_timer, K_SECONDS(60), K_SECONDS(60));
185
186     LOG_INF("Periodic uplinks started (60s interval)");
187
188     /* Main loop */
189     while (1) {
190         k_sleep(K_SECONDS(10));
191     }
192
193     return 0;
194 }

```

5.2 TTN Payload Decoder

Add this decoder function in your TTN application:

```

1 function decodeUplink(input) {
2     var data = {};
3
4     // Decode temperature (bytes 0-1, int16, x10)
5     var temp_raw = (input.bytes[0] << 8) | input.bytes[1];
6     if (temp_raw > 32767) temp_raw -= 65536; // Handle negative
7     data.temperature = temp_raw / 10.0;
8
9     // Decode humidity (byte 2, uint8)
10    data.humidity = input.bytes[2];
11
12    // Decode battery voltage (bytes 3-4, uint16, mV)
13    data.battery_voltage = ((input.bytes[3] << 8) |
14                           input.bytes[4]) / 1000.0;
15
16    // Sequence counter (byte 5)
17    data.sequence = input.bytes[5];
18
19    return {
20        data: data,
21        warnings: [],
22        errors: []
23    };
24 }

```

6 Expected Output

```

1 [00:00:10.570] <inf> lab_1_2: Network joined successfully!
2 [00:00:10.571] <inf> lab_1_2: Encoded: Temp=20.7 C , Hum=65%, Vbat=3625
   mV
3 [00:00:10.572] <inf> lab_1_2: Uplink sent (6 bytes)
4 [00:00:10.573] <inf> lab_1_2: Payload: 00 cf 41 0e 29 00
5 [00:01:10.571] <inf> lab_1_2: Encoded: Temp=21.2 C , Hum=62%, Vbat=3638
   mV
6 [00:01:10.572] <inf> lab_1_2: Uplink sent (6 bytes)
7 [00:01:11.234] <inf> lab_1_2: =====
8 [00:01:11.235] <inf> lab_1_2: DOWNLINK RECEIVED
9 [00:01:11.236] <inf> lab_1_2: =====
10 [00:01:11.237] <inf> lab_1_2: Port: 2
11 [00:01:11.238] <inf> lab_1_2: RSSI: -52 dBm
12 [00:01:11.239] <inf> lab_1_2: SNR: 9 dB
13 [00:01:11.240] <inf> lab_1_2: Length: 1 bytes
14 [00:01:11.241] <inf> lab_1_2: Data: 01
15 [00:01:11.242] <inf> lab_1_2: Command: LED ON

```

7 Deliverables

1. Source code with uplink/downlink implementation
2. Serial log showing bidirectional communication

3. TTN console screenshot with decoded payload
4. Test downlink from TTN showing device receives it

8 Additional Challenges

8.1 Challenge 1: Payload Compression

Reduce payload to 4 bytes using:

- Temperature: 1 byte (range -40 to +85°C)
- Humidity: 1 byte (0-100)
- Battery: 1 byte (2.0V to 4.2V)
- Counter: 1 byte

8.2 Challenge 2: Downlink Actuator

Control actual LED based on downlink commands.